Package 'lorps'

November 7, 2026

Type Package

Title Low-Rank Probit Stick-Breaking Multiple Imputation

Version 0.1.0

Description Provides a Bayesian nonparametric approach to multiple imputation of missing values in high-dimensional data matrices. The method combines flexible mixture modeling with a low-rank structure to borrow information across rows and columns, allowing for complex data distributions while accounting for uncertainty in the missing values.

License GPL-3

Encoding UTF-8

Depends R (>= 4.1.0)

Imports Rcpp, RcppTN, parallel, parallelly, stats, utils

LinkingTo Rcpp, RcppArmadillo (>= 14.6.0-1), RcppTN

Suggests mitools

SystemRequirements C++17

NeedsCompilation yes

Contents

lorps-package 1
complete_lorps 2
lorps_mi 4
lorps_suggest_K 8
print.lorps_mi 9

Index **10**

`lorps-package` *Low-Rank Probit Stick-Breaking Multiple Imputation*

Description

The main function `lorps_mi` fits a flexible normal-mixture model to an incomplete numeric matrix, with information shared through latent row, column, and mixture-component factors. The package compiles the sampler and provides initialization, optional rank selection, and compact storage of retained missing-entry draws. Use `complete_lorps` to construct complete datasets and `lorps_suggest_K` to evaluate a sufficient truncation level from the manuscript's prior bound.

Author(s)

Bhargob Kakoty, Thierry Chekouo, Eric F. Lock

`complete_lorps` *Construct Completed Matrices from a LoRPS Fit*

Description

Reconstructs completed datasets on demand, preserving observed values and the joint alignment of the imputed entries in each retained draw.

Usage

```
complete_lorps(object, action = 1L)
```

Arguments

`object`	An object returned by `lorps_mi`.
`action`	One or more retained-draw numbers from 1 to `object$n_imputations`; 0 for the original incomplete matrix; `"all"` for all completed matrices; or `"mean"` for a single matrix with missing entries replaced by their posterior means.

Details

Each draw number identifies one entire column of `object$imputations`. Do not independently permute draw columns across missing entries: that would destroy their joint alignment. Requesting `"all"` materializes all complete matrices and may need much more memory than the compact fit. For memory-efficient downstream analysis, analyze one completed draw at a time and retain only its estimates and covariance matrix.

The `"mean"` completion is a point summary and does not propagate imputation uncertainty. For nonlinear quantities, calculate the quantity within each completed draw and then summarize across draws.

For compatible downstream models, an ordinary list of completed data frames can be passed to `mitools::imputationList`; complete-data estimates and variances can then be combined with `mitools::MIcombine`. This function does not fit downstream models or pool estimates itself.

Value

A numeric matrix if one draw, 0, or `"mean"` is requested. A named list of numeric matrices for multiple draw numbers or `"all"`.

See Also

`lorps_mi`

Examples

```
# Generate a complete matrix with a low-rank underlying signal.
set.seed(10)

N <- 30
Q <- 20
signal_rank <- 2

U <- matrix(rnorm(N * signal_rank), nrow = N)
V <- matrix(rnorm(Q * signal_rank), nrow = Q)

signal <- tcrossprod(U, V)
X_true <- signal + matrix(rnorm(N * Q, sd = 0.3), nrow = N)

# Randomly mask 20 percent of the entries.
missing_index <- sort(
  sample.int(length(X_true), size = floor(0.20 * length(X_true)))
)

X <- X_true
X[missing_index] <- NA_real_

# Fit LoRPS with a fixed tensor rank.
# These are illustrative settings; convergence should be assessed.
fit <- lorps_mi(
  X,
  R = 2,
  K = 10,
  M = 2000,
  burn_in = 1000,
  thin = 5,
  seed = 12,
  verbose = FALSE
)

# Construct one completed matrix.
one <- complete_lorps(fit, action = 1)

# Construct a list containing the first and fifth completed matrices.
two <- complete_lorps(fit, action = c(1, 5))

# Replace missing entries with their means across imputations.
point_estimate <- complete_lorps(fit, action = "mean")

# Retrieve the original incomplete matrix.
original <- complete_lorps(fit, action = 0)

# Evaluate point accuracy using only the artificially masked entries.
true_missing <- X_true[fit$missing_index]
imputed_mean <- point_estimate[fit$missing_index]

nmse <- mean((imputed_mean - true_missing)^2) / var(true_missing)
print(c(NMSE = nmse))

# Check that the original matrix and observed values are preserved.
stopifnot(
```

```
  identical(original, fit$X),
  length(two) == 2L,
  identical(one[!is.na(X)], X[!is.na(X)]),
  identical(point_estimate[!is.na(X)], X[!is.na(X)])
)

## Not run:
# Construct all completed matrices.
# For large datasets, processing one completion at a time uses less memory.
all_completed <- complete_lorps(fit, action = "all")
length(all_completed)

## End(Not run)
```

`lorps_mi` *Multiple Imputation with a Low-Rank Probit Stick-Breaking Model*

Description

Performs Bayesian multiple imputation for incomplete numeric matrices using a Low-Rank Probit Stick-Breaking Process (LoRPS). Entry-specific normal mixtures accommodate flexible distributions, while a low-rank latent structure shares information across rows and columns. The tensor rank can be fixed or selected using held-out predictive likelihood.

Usage

```
lorps_mi(X, R = NULL, K = 20, CV = is.null(R),
         cand_rank_list = 1:15,
         M = 2000, burn_in = 1000, thin = 1,
         mus_init = NULL, sigma2s_init = NULL, C_ij_init = NULL,
         tau2 = 10000, cv_control = list(),
         seed = NULL, verbose = TRUE)
```

Arguments

`X`	A nonempty real numeric matrix, with individuals in rows and variables in columns. Missing values are `NA` or `NaN`. Infinite values and entirely missing matrices are not supported. A complete matrix is also accepted. No automatic scaling, transformation, or restriction of imputations to the observed range is applied.
`R`	A positive integer tensor rank parameter, or `NULL`. With the defaults, `NULL` requests cross-validation (CV), and an explicit value requests a fixed-rank fit. If `CV = FALSE` and `R = NULL`, rank 5 is used. Explicit `CV = TRUE` overrides a supplied `R`, with a warning.
`K`	Number of normal mixture components in the finite stick-breaking representation, at least 2. The default is 20. The final component receives the remaining stick mass. See `lorps_suggest_K` for guidance based on a prior truncation bound.
`CV`	Logical. Whether to select `R` by maximizing the cross-validated log posterior predictive likelihood (LPPL). The default is `is.null(R)`.

`cand_rank_list` Positive integer candidate ranks. Used only for CV; duplicate values are removed and the values are sorted. Larger lists require more sampler fits. Exact ties select the smaller rank.

`M`, `burn_in`, `thin` Total sampler iterations, burn-in setting, and thinning interval. `M` and `thin` must be positive integers; `burn_in` must be a nonnegative integer. Both `M` and `burn_in` must be multiples of `thin`. Under the sampler's saving convention, `(M - burn_in) / thin - 1` draws are retained, and this number must be at least 2. The defaults give 999 draws. These settings do not guarantee convergence.

`mus_init`, `sigma2s_init`, `C_ij_init`

Optional starting values for the component means, variances, and assignments. Supply all three together or leave all three as `NULL`. By default, k-means-based initialization is used, with missing entries temporarily mean-filled to construct starting values. The first two arguments must be vectors of length `K`, containing finite means and positive finite variances. `C_ij_init` must be a matrix with `dim(X)` and zero-based integer labels `0:(K - 1)`, including positions missing in `X`. Custom starting values apply only to the final fit; CV fits are initialized using their own training data.

`tau2` Positive finite variance of the normal prior $\mu_k \sim \mathcal{N}(0, \tau^2)$ on each component mean. The default is 10000. The remaining prior distributions are fixed and described under Model.

`cv_control` A named list overriding any CV defaults:

`nfold = 5` Number of disjoint validation subsets, at least 2.

`holdout_fraction = 0.05` Fraction of originally observed entries in each validation subset, rounded down to an integer count.

`M = 1000, burn_in = 500, thin = 1` CV sampler settings, giving 499 retained draws by default. The same divisibility and retained-draw checks apply as for the final fit.

`cores = NULL` Automatic use of up to five workers, capped by the number of subsets and `parallelly::availableCores()`. A positive integer supplies a different upper limit; 1 forces serial CV. A PSOCK cluster is used across platforms. If cluster startup fails, CV falls back to serial execution with a warning.

`seed` Optional nonnegative integer. An explicit seed is local to this call, and the previous R random-number state is restored on exit. With `NULL`, the current R random-number stream is used.

`verbose` Logical. Whether to show progress messages and the final sampler's iteration output. CV sampler output is suppressed.

Details

Model: Let Θ denote the model parameters. Conditional on these parameters, the matrix entries are independent, with

$$X_{ij} \mid \Theta \sim \sum_{k=1}^{K} w_{ijk}\, \mathcal{N}(\mu_k, \sigma_k^2).$$

The component means and variances are shared across the matrix, whereas the mixture weights vary by row and column. The weights follow $w_{ijk} = V_{ijk} \prod_{\ell<k}(1-V_{ij\ell})$, where $V_{ijk} = \Phi(Z_{ijk})$ for $k < K$, and $V_{ijK} = 1$. Here, Φ is the standard normal cumulative distribution function. The latent tensor has a CP/PARAFAC representation with R terms:

$$Z_{ijk} = \sum_{r=1}^{R} Z_1[i,r] Z_2[j,r] Z_3[k,r].$$

The factor matrices link rows, columns, and mixture components. Low-rank structure is therefore imposed on the latent variables governing the weights, rather than directly on the data matrix.

Factor entries have independent standard normal priors. Independently, component means have normal priors with mean zero and variance `tau2`, and component variances have inverse-gamma priors with fixed shape 2 and scale 1.

Mixture truncation: K: `K` is the number of normal components used to approximate the infinite mixture. The helper `lorps_suggest_K()` provides guidance using a bound on the discrepancy between the finite and infinite prior joint marginal densities. Its result must be supplied explicitly as `K`; `lorps_mi()` does not select `K` automatically.

Tensor rank: R: `R` controls the complexity of the latent factorization. When `CV = TRUE`, the model is fitted at each candidate rank with selected observed entries temporarily held out. For each validation set, its joint held-out likelihood is averaged over posterior draws, and the logarithms of these averages are summed across validation sets. This gives the cross-validated log posterior predictive likelihood (LPPL) score. The rank with the largest score is selected, and the model is refitted using all originally observed entries.

By default, five disjoint validation sets each contain approximately 5% of the observed entries; they do not partition all observed entries into five equal folds. The same sets are used for every candidate rank, and genuinely missing entries are never scored. A larger rank search requires additional fits; a fixed rank avoids this search.

Multiple imputations: Posterior computation uses Markov chain Monte Carlo (MCMC). Each retained iteration supplies one joint draw of all missing entries, which, together with the unchanged observed entries, forms a completed dataset. The draws are stored compactly in `imputations`; completed matrices are constructed using `complete_lorps()`. The number retained is reported in `n_imputations` and depends on `M`, `burn_in`, and `thin`.

Value

An object of class `lorps_mi`, with:

`X` The numeric input matrix, preserving row and column names.

`missing_index` Linear indices of missing entries in R's column-major order, matching `which(is.na(X))`.

`imputations` A matrix with one row per missing entry and one column per retained joint draw. With complete input it has zero rows.

`mu_draws` Component-mean draws, with `K` rows and columns corresponding to the same retained iterations as `imputations`. Other model-parameter draws are not returned.

`R, K, n_imputations` Selected or supplied rank, truncation level, and actual number of retained draws.

`iterations` Retained zero-based sampler iteration labels.

`cv` `NULL` for a fixed-rank fit; otherwise, a list: `results` gives one row per rank, validation scores, their total and mean, and a selection flag; `fold_scores` is the rank-by-subset score matrix; `folds` contains the held-out linear indices; `n_observed` and `fold_size` record sizes; `controls` contains resolved settings; `cores_used` is the actual worker count; and `seeds` contains initialization and sampler task seeds.

`settings` Final sampler settings, prior variance, seed, rank-selection method, and a prior joint-density truncation upper bound capped at the maximum possible L1 distance of 2.

`seeds, rng_kind, call` Final initialization and sampler seeds, R random-number generator configuration, and the function call.

No accuracy statistic is computed for genuinely missing values. Completed datasets can be reconstructed with `complete_lorps`.

References

Kakoty, B., Chekouo, T., and Lock, E. F. *Bayesian Nonparametric Multiple Imputation for High-Dimensional Matrices*. Manuscript.

See Also

complete_lorps, lorps_suggest_K

Examples

```
# Generate a complete matrix with a low-rank underlying signal.
set.seed(10)

N <- 30
Q <- 20
signal_rank <- 2

U <- matrix(rnorm(N * signal_rank), nrow = N)
V <- matrix(rnorm(Q * signal_rank), nrow = Q)

signal <- tcrossprod(U, V)
X_true <- signal + matrix(rnorm(N * Q, sd = 0.3), nrow = N)

# Randomly mask 20 percent of the entries.
missing_index <- sort(
  sample.int(length(X_true), size = floor(0.20 * length(X_true)))
)

X <- X_true
X[missing_index] <- NA_real_

# Fit LoRPS with a fixed tensor rank.
# These are illustrative settings; convergence should be assessed.
fit <- lorps_mi(
  X,
  R = 2,
  K = 10,
  M = 2000,
  burn_in = 1000,
  thin = 5,
  seed = 12,
  verbose = FALSE
)

fit
dim(fit$imputations)

# Average the imputation draws for each missing entry.
imputed_mean <- rowMeans(fit$imputations)

# Match the true values to the ordering of the stored imputations.
true_missing <- X_true[fit$missing_index]

# NMSE: mean squared prediction error divided by
# the sample variance of the true masked values.
nmse <- mean((imputed_mean - true_missing)^2) / var(true_missing)
```

```
print(c(NMSE = nmse))

# Obtain one completed dataset or a posterior-mean completion.
completed <- complete_lorps(fit, action = 1)
completed_mean <- complete_lorps(fit, action = "mean")

# Observed entries remain unchanged.
stopifnot(identical(completed[!is.na(X)], X[!is.na(X)]))

## Not run:
# Alternatively, select the tensor rank by predictive-likelihood CV.
fit_cv <- lorps_mi(
  X,
  K = 10,
  cand_rank_list = 1:3,
  cv_control = list(cores = 1),
  M = 2000,
  burn_in = 1000,
  thin = 5,
  seed = 12,
  verbose = FALSE
)

fit_cv$R
fit_cv$cv$results

imputed_mean_cv <- rowMeans(fit_cv$imputations)
true_missing_cv <- X_true[fit_cv$missing_index]

nmse_cv <- mean((imputed_mean_cv - true_missing_cv)^2) /
  var(true_missing_cv)

print(c(NMSE = nmse_cv))

## End(Not run)
```

`lorps_suggest_K` *Choose a Truncation Level from the LoRPS Prior Bound*

Description

Calculates a sufficient finite truncation level for a specified upper bound on the L1 discrepancy between the finite and infinite prior joint marginal densities in Theorem 1 of the LoRPS manuscript.

Usage

```
lorps_suggest_K(N, Q, tolerance = 1e-3)
```

Arguments

`N`	Positive integer number of rows.
`Q`	Positive integer number of columns.
`tolerance`	Target L1 bound, strictly between 0 and 2.

Details

The theorem gives

$$\|m^{(K)} - m^{(\infty)}\|_1 \le 2NQ\,2^{-(K-1)}.$$

Consequently, the helper returns

$$\max\{2,\ 1 + \lceil \log_2(2NQ/\epsilon) \rceil\},$$

where ϵ is `tolerance`. The final stick is forced to one. The bound concerns prior joint marginal densities under the stated model.

The package default `K = 20` follows the manuscript's practical recommendation. It is independent of this helper; the helper does not silently alter the fit. In a 348 by 574 matrix, `K = 30` makes the raw bound approximately 0.000744.

Value

An integer `K`, at least 2.

References

Kakoty, B., Chekouo, T., and Lock, E. *Bayesian Nonparametric Multiple Imputation for High-Dimensional Matrices*. Manuscript.

Examples

```
lorps_suggest_K(348, 574)  # 30
K <- lorps_suggest_K(100, 50, tolerance = 0.01)
2 * 100 * 50 * 2^(-(K - 1))
```

---

print.lorps_mi *Print a LoRPS Multiple-Imputation Fit*

Description

Displays matrix dimensions, the number of missing entries, the selected or fixed rank, the number of components, and the retained draws.

Usage

```
## S3 method for class 'lorps_mi'
print(x, ...)
```

Arguments

`x`	An object of class `lorps_mi`.
`...`	Further arguments; currently unused.

Value

The input object, invisibly.

See Also

`lorps_mi`

Index

∗ **package**
lorps-package, 1

complete_lorps, *1*, 2, *6, 7*

lorps *(*lorps-package*)*, 1
lorps-package, 1
lorps_mi, *1, 2*, 4, *9*
lorps_suggest_K, *1*, *4*, *6, 7*, 8

print.lorps_mi, 9